

# Resource Contracts for Completing Optimistic Computations

ANONYMOUS AUTHOR(S)

An optimistic computation can retry, compute under protection, or validate a prepared result. We ask how a scheduler can retain optimal protected work at every unknown interference budget while choosing other job priorities. For finite whole-kernel jobs with persistent outputs, an exact residual safety filter needs only completed/current information, with constant-time mode queries. Every caller following its immediate preparation protocol has worst total work within  $(3r + 2)/(2r + 2)$  of the best general program satisfying the same universal call and protection contract, on every dependency DAG;  $r$  is the spare-call allowance. The factor is sharp over all permitted callers, equals  $5/4$  at  $r = 1$ , and equals one at  $r = 0$ . An exact independent-job curve supplies the benchmark for sharpness. We also characterize when fixed mismatch and validation charges require future fee information. Conditional Java correspondence and native Linux/Roslyn studies delimit the work, observation and call assumptions. The result bounds the body-work cost of scheduling freedom; the native studies establish neither general cost transfer nor latency gains.

CCS Concepts: • **Software and its engineering** → **Concurrent programming structures**.

Additional Key Words and Phrases: concurrent programming, retries, resource guarantees, scheduling, online policies

## 1 Introduction

A library that completes optimistic computations must decide when to retry, when to compute under protection, and when to validate a prepared result before protected recomputation. These established mechanisms [33, 44] leave a contract question: what total work does a fixed failed-call allowance and optimal protection at every unknown write budget require?

These are different resources. One unit-work job illustrates the choice. Retrying once and then computing under protection uses at most two bodies but protects a body after one interfering write. Preparing again before a guarded cached completion instead protects no body after one write, yet can execute three bodies after two writes. Both have two logical completion calls. A retry counter alone therefore does not determine the work contract.

Linux's rename-lock pathname loop [7, 8] allows two body attempts; our cached variant permits a third to reduce protected rebuilding. It relaxes that source bound. Roslyn's immutable-solution loop has lazy work and guarded merges that prevent fixed-work transfer, and its dense-arrival measurements favor two modes. A logical call is therefore neither a duration nor necessarily a native attempt.

We study finite dependent whole-kernel jobs with persistent outputs and hidden total interference. Retry slack  $r$  allows  $n + r$  completion calls for  $n$  jobs. After  $r$  failures, cached operations attain optimal protection: the sum of the largest  $(B - r)_+$  weights under at most  $B$  writes. We ask what *keeping that optimal curve at every budget* costs, and how other completing modes can preserve it.

The results answer three connected design questions.

- (1) *What total work does the contract force?* Independent jobs admit an exact least worst-work curve, attained simultaneously at all budgets by one sorted policy. The lower bound covers every program in the declared interface, including retained records and paid early preparation.
- (2) *How much can other scheduling priorities cost?* On every DAG, an exact filter exposes all completing modes preserving the residual protection contract. Every caller obeying its immediate preparation protocol has worst body work within  $\alpha_r = (3r + 2)/(2r + 2)$  of the full optimal-protection program class. The factor is sharp over all callers, and is  $5/4$  for

Table 1. Notation for the body-work contract. Charged extensions introduce their own operation prices.

| Symbol                         | Meaning                                                                                                         |
|--------------------------------|-----------------------------------------------------------------------------------------------------------------|
| $J, D, S$                      | All jobs, completed ideal, and unfinished jobs $S = J \setminus D$ .                                            |
| $w_i, \Omega, \Omega_k(D)$     | Positive work of one whole kernel, total job work, and sum of the $k$ largest completed works.                  |
| $r, B$                         | Allowed failed calls and hidden bound on external writes; both are integers.                                    |
| $Q, W, L$                      | Completion calls, all executed kernel work, and its protected part. Preparations add work but no call.          |
| $k, \ell$                      | Observed cached mismatches and protection incurred after the cheap phase.                                       |
| $\mathcal{V}_r, \mathcal{A}_r$ | All programs meeting the simultaneous call/protection contract, and the composed immediate-preparation callers. |
| $F_J(B)$                       | Least worst total work over $\mathcal{V}_r$ at budget $B$ ; the comparator still must satisfy every budget.     |

one spare call. The filter needs no future weight or edge: queries take  $O(1)$  operations and updates  $O(\log n)$ .

- (3) *When does that interface stop being exact?* Arbitrary mismatch premiums admit an exact region using completed mismatch costs and future premiums. The DAG still affects only readiness. We also solve three fixed actual outcome prices; costly validation changes the required future information. A reachable identical-prefix pair proves that hiding arbitrary future premiums loses exactness.

For independent works 8 and 16 with no spare calls, one universally optimal-protection policy requires worst  $(W, L) = (40, 16)$  at  $B = 1$ . If  $B = 1$  is supplied, cache the 8-unit job first: on match finish 16 fresh; on mismatch the write budget is exhausted, so prepare/cache 16. This attains  $(32, 16)$ . The difference measures the price of the simultaneous protection contract, not the price of merely omitting a budget argument.

Top-budget sums, adaptive testing and safety filters are inherited; our exact interface bounds the body-work cost of other scheduling priorities. For independent works  $(8, 4, 2, 1)$  and  $r = 1$ , descending cheap order then caching costs 38 at a saturated budget; ascending costs 31 with the same protection. The factor bounds such choices without optimizing each DAG. Java wrappers execute the contract using existing synchronization. An actual client requiring its full guarantee remains unestablished. Section 10 compares closest work. Earlier same-project snapshots [22, 23] cover retained guards, supplied-budget frontiers and charged complexity; they are neither independent prior work nor additional validation.

## 2 Completion Interfaces and Resource Contracts

Write  $\mathbb{N} = \{0, 1, \dots\}$ . Throughout, write budgets  $B, b$  and retry allowances  $r$  lie in  $\mathbb{N}$ . Jobs  $J = \{1, \dots, n\}$  form a finite DAG,  $n \geq 1$ . Job  $i$  is ready when every predecessor in  $\text{Pred}(i)$  has completed. Each completion saves an immutable output despite later writes. Completion requires one transformation per job, without a simultaneous snapshot. Policies are deterministic and causal.

Each ready job has a pure, opaque whole kernel on its own input and exact saved predecessor outputs. Every execution performs work  $w_i > 0$ , inside or outside protection. Partial/incremental evaluation, preparation before readiness and external completion are excluded. A prepared result certifies those inputs; subsequent map reads cannot reconstruct saved outputs.

*State and transition notation.* Write  $\sigma = (x, \mathcal{M}, \mathcal{R}, \lambda)$  for live own-input identities, completed outputs, a finite snapshot/record store and program locals. A record  $\rho = (i, s, \mathbf{u}_i, z)$  certifies a completed kernel execution on captured identity  $s$  and saved predecessor tuple  $\mathbf{u}_i = \mathcal{M}|_{\text{Pred}(i)}$ , returning  $z$ . Copies retain this certificate; fabrication or retagging requires new computation. Define

pub<sub>*i*</sub>(*z*) to assign  $x_i := z$ , save  $\mathcal{M}(i) := z$  and complete *i*. Only publishing completion changes an unfinished live key through a foreground action. Table 2 gives effects, observations and charges;  $m = [s = x_i]$  is exact identity comparison. A failed cheap call clears selection, while its retained record copies remain available.

*A turn and its write gate.* The program selects an action *a*, fixing its job and mode. The environment then chooses a finite legal write word  $\omega$  before that action's effect:

$$\sigma \xrightarrow{\omega} \widehat{\sigma} \xrightarrow{a/o, \delta} \sigma'.$$

Here *o* is the returned observation and  $\delta = \Delta(Q, W, L)$ . Each external assignment changes its own  $x_i$  and leaves  $\mathcal{M}$  immutable. At every finite history an unfinished target admits an identity fresh to all its prior identities, records and raw snapshots, including snapshots predating the charged interval. The adversary may choose never to reuse one; finiteness alone does not provide this supply.

After the gate, record selection uses only the already-computed prior store. A new unprotected preparation, even if paid, is a separate action with a later comparison gate. Completing comparison/computation/publication is indivisible to writes; every guard ends on return. Inspections honestly return live entries. Cached completion reveals its output and comparison Boolean; retained values, reference equality and local state remain available. Clocks, allocation identifiers and work counters are not observations. Section 7 separately restricts cached observations.

*Preparation and charging.* General programs may prepare early after readiness, using immutable own snapshots and saved parents; raw snapshots may be old. Immediate preparation captures and computes only for the next call. A pure unprotected body has no live interaction: at completed-operation boundaries, contract it with its returned record and charge, and replay intervening writes before the next action. Section 8 accounts for pending writes without placing a completed result at an unfinished source prefix.

The common start precedes all computed initialization. *W* charges every kernel, including discarded/early computations; *L* charges its protected part. *Q* counts validation/publication/completion calls, including failures, but not preparations or writers. Waiting, common call overhead and nonkernel allocation are outside these coordinates. Write  $\Omega_k(T)$  for the sum of the largest  $\min(k, |T|)$  works,  $\Omega(T)$  for total work, and  $\Omega_0 = 0$ ; omitted sets mean *J*. These aggregate objects are inherited [13, 14].

Table 2. Transitions after the write gate. A selected record is  $\rho = (i, s, \mathbf{u}_i, z)$ ;  $m = [s = x_i]$  and  $v_i = (1 - m)w_i$ . Kernel/completion actions require a ready unfinished job; inspection may read any key.  $z'$  is a new kernel result.

| Action                   | Effect                                                                                              | Observation    | $\Delta(Q, W, L)$ |
|--------------------------|-----------------------------------------------------------------------------------------------------|----------------|-------------------|
| Inspect <i>i</i>         | No state change                                                                                     | $x_i$          | $(0, 0, 0)$       |
| Prepare <i>i</i>         | Run kernel on captured $(s, \mathbf{u}_i)$ ; add $\rho$ to $\mathcal{R}$                            | $\rho$         | $(0, w_i, 0)$     |
| Cheap <i>i</i> , $\rho$  | If $m$ , pub <sub><i>i</i></sub> ( <i>z</i> ); otherwise clear selection                            | $m$            | $(1, 0, 0)$       |
| Fresh <i>i</i>           | Run kernel on $(x_i, \mathbf{u}_i)$ ; pub <sub><i>i</i></sub> ( $z'$ )                              | $z'$           | $(1, w_i, w_i)$   |
| Cached <i>i</i> , $\rho$ | If $m$ , pub <sub><i>i</i></sub> ( <i>z</i> ); else run kernel and pub <sub><i>i</i></sub> ( $z'$ ) | Output and $m$ | $(1, v_i, v_i)$   |

Free copying, selection and local actions modify only  $\mathcal{R}, \lambda$ . Two modes allow cheap/fresh; three add cached. “Cheap” refers to kernel work. Direct locking, retained guards, cross-job callbacks and outside completion are excluded. Selected operations terminate. Admissibility requires completion

of every job after finitely many actions on each admitted play; infinite free actions fail it. No uniform action bound over all environments is assumed.

*Strategies and admissibility.*  $B$  bounds external writes;  $r$  permits extra failed calls beyond  $n$  completions. An informed strategy is  $P(B, h)$ ; an unaware one is  $P(h)$ . Let  $\mathcal{E}_b$  contain gate strategies using at most  $b$  writes on every play. For mode family  $j$ ,  $\mathcal{P}_j(B, r)$  requires completion with  $Q \leq n + r$  against every  $E \in \mathcal{E}_B$ . In contrast, one  $P \in \mathcal{U}_j(r)$  must satisfy that cap for every  $b \geq 0$  and  $E \in \mathcal{E}_b$ , without receiving  $B$ . The gate adversary knows the deterministic program and its prior store. A fixed adverse play can be scripted at gates; no wall-clock-oblivious or randomized value follows. Its finite write count also bounds a capped environment preserving that play. Worst protection is  $\sup_{E \in \mathcal{E}_B} L(P, E)$ .

LEMMA 1 (FRESH-WRITE AND FINITE-ADVERSARY FACTS). *In Section 2's game, a fresh own-key write at a selected  $i$ -operation's gate invalidates every retained  $i$ -record. Any concrete prefix has both a no-write and a fresh-write continuation whenever that extra write respects the environment's budget. An adversary bounded by  $b$  writes on every play forces any  $P \in \mathcal{P}_j(b, r)$  to finish every job, regardless of free inspections or record retention.*

PROOF. A fresh identity differs from every retained target identity. Copying/selection cannot change that, and all observations and job/mode choices agree until the gate. The bounded adversary belongs to  $\mathcal{E}_b$ ; leaving any job unfinished violates admissibility even with infinitely many free actions. Thus every job completes through Table 2. No unbounded adversary is truncated.  $\square$

LEMMA 2 (WORK CHARGED BY USEFUL COMPARISON WRITES). *From the charged common start, consider a complete execution in which the adversary writes only fresh own-key identities at comparisons whose zero-write continuation would match. Let  $H$  be the multiset of resulting cheap failures and  $D$  the set of resulting cached mismatches. Then*

$$W \geq \Omega + \sum_{h \in H} w_{i(h)} + \sum_{i \in D} w_i.$$

*Already-stale failures may consume calls, but need no extra charge in this inequality.*

PROOF. Let  $\mathcal{K}$  be the set of completed kernel invocations since the common charged start. Give each original prepared record a proof-only creator  $\iota(\rho) \in \mathcal{K}$ ; copying preserves that creator. Its job and captured inputs never change. These labels are not observations supplied to the program.

For each useful invalidated comparison  $e$ , let  $\rho_e$  be the record selected by the *zero-write continuation of that same call*. Define  $\phi(e) = \iota(\rho_e)$ . Although the failure continuation may select another record,  $\rho_e$  was already in the pre-gate store, so  $\phi(e)$  was executed and charged on the actual run. This defines  $\phi$  for both useful cheap failures and useful cached mismatches.

We show that  $\phi$  is injective. Different jobs have different kernel invocations. At one job, the fresh write following  $\rho_e$  changes the live input to an identity absent from all earlier captures. Subsequent useful comparisons must select records matching later live inputs. The fresh-identity supply of Section 2 lets this writer never restore an earlier identity; an unfinished foreground action cannot restore one either. Thus their creators differ from  $\phi(e)$ , including all copies of that invocation's record.

For each completed job  $i$ , choose the invocation  $\psi(i)$  supplying its final completion: the protected invocation for fresh/stale cached, or the creator of the matching published record. These  $n$  invocations are distinct across jobs. At job  $i$ , every useful invalidation precedes completion and invalidates the input of its charged creator. Hence  $\psi(i)$  is distinct from all  $\phi(e)$  at that job. A useful

cached mismatch therefore contributes two distinct invocations:  $\phi(e)$  for its preparation and  $\psi(i)$  for its protected recomputation. Already-stale calls supply no additional  $\phi$  charge.

The images of  $\phi$  and  $\psi$  are disjoint subsets of  $\mathcal{K}$ . Each invocation at  $i$  costs  $w_i$ , giving the inequality. This argument charges early work and old-snapshot computations at their actual creation; it does not treat a suffix with prepaid records as a free new start.

□

### 3 Universal Call and Protected-Work Guarantees

**THEOREM 1 (OPTIMAL PROTECTION ON EVERY DAG).** *For integers  $B, r \geq 0$ , the least worst protected work among programs supplied with  $B$  and completing with  $Q \leq n + r$  is  $\Omega_{(B-r)_+}$ . One policy attains this entire curve without  $B$ : immediately prepare before cheap calls until  $r$  failures or completion, then immediately prepare/cache every remaining job in any ready order.*

**PROOF.** Each failed cheap call needs a distinct write in its current-capture/comparison interval. There are at most  $r$  failures and  $n$  completing calls. After the switch at most  $(B - r)_+$  distinct jobs mismatch, giving the upper bound; at  $B \leq r$  no protected body is needed.

For the lower, target the  $k = \min((B - r)_+, n) > 0$  heaviest jobs. Count target cheap failures  $f$  and completed targets  $t$ . At their comparison gates, install a fresh identity while  $f \leq r$  and  $t < k$ ; stop permanently at  $f = r + 1$  or  $t = k$ . Every write causes a failure or cached completion, so at operation boundaries its count is  $v \leq f + t$ . Before another write,  $v + 1 \leq f + t + 1 \leq r + k \leq B$ , even on inadmissible programs. Thus this is an unconditionally bounded writer. The call cap excludes  $r + 1$  cheap failures; Lemma 1 forces every target to complete. Its comparison cannot match, so all target bodies are protected and  $L \geq \Omega_k$ . Readiness may delay a target but cannot remove its completion obligation. Free inspections and retained records do not evade its fresh gate identity. At  $k = 0$  nonnegative work suffices. □

With only cheap/fresh modes, the analogous supplied-budget minimum is 0 at  $B \leq r$  and  $\Omega$  otherwise; requiring one universally call-bounded policy changes the boundary to  $B \geq r$ . After  $r$  failures, an unaware policy must still tolerate another write. The archive gives those two-mode proofs [23].

### 4 The Total Work Needed for Optimal Protection

We minimize worst total work while keeping optimal protection at every budget. Separate  $W, L$  suprema may have different witnesses; they do not determine  $\sup(W + \lambda L)$ .

Let  $\mathcal{V}_r$  contain the policies in  $\mathcal{U}_3(r)$  satisfying  $L(P, E) \leq \Omega_{(b-r)_+}$  for every  $b \geq 0$  and  $E \in \mathcal{E}_b$ . This requires one *simultaneous optimal-protection curve*; it is stronger than universal completion. Write  $W_P(B) = \sup_{E \in \mathcal{E}_B} W(P, E)$ .

**LEMMA 3 (CAPPING A REALIZED PLAY).** *For  $P \in \mathcal{V}_r$ , a finite prefix containing  $d$  actual writes has protected work at most  $\Omega_{(d-r)_+}$ .*

**PROOF.** Continue the prefix without writes. Admissibility forces completion. Cap that environment unconditionally at  $d$ : induction preserves each action and realized write, and there are no later writes to remove. The resulting environment belongs to  $\mathcal{E}_d$  and preserves the complete play. Its total protected work satisfies the bound; nonnegative costs give the prefix bound. The program need not observe  $d$ . □

THEOREM 2 (LEAST TOTAL-WORK CURVE UNDER SIMULTANEOUS OPTIMAL PROTECTION). *For independent jobs with sorted positive works  $a_1 \leq \dots \leq a_n$ ,*

$$\inf_{P \in \mathcal{V}_r} W_P(B) = F_r(B) := \begin{cases} \Omega + Ba_n, & B \leq r, \\ \Omega + \max_{1 \leq m \leq \min(B-r, n)} \{ra_{n-m+1} + \Omega_m\}, & B > r. \end{cases}$$

*One policy attains this entire curve: process jobs in ascending work order, freshly prepare before each cheap call until  $r$  cheap failures, and freshly prepare before each cached completion thereafter. It observes neither  $B$  nor cached comparison results. Sorting, prefix sums and prefix maxima construct the curve in  $O(n \log n)$  arithmetic/comparison operations, representing its initial linear segment analytically.*

PROOF. *Upper bound.* Each bad comparison needs a write in its distinct preparation-to-comparison interval. There are at most  $r$  cheap failures; cached mismatches occur only thereafter, at distinct jobs. Thus  $Q \leq n + r$  and  $L \leq \Omega_{(B-r)_+}$  at every budget. When  $B \leq r$ , at most  $B$  preparations are lost, each costing at most  $a_n$ .

For  $B > r$ , suppose cached mode starts at rank  $j$  and has  $m \geq 1$  mismatches. All failed preparations were at ranks at most  $j$  and the  $m$  mismatched jobs lie in the suffix, so  $j \leq n - m + 1$ . Duplicate work is at most  $ra_j + \Omega_m \leq ra_{n-m+1} + \Omega_m$ , with  $m \leq B - r$ . Without a mismatch it is at most  $ra_n$ , below the  $m = 1$  term. For each permitted  $m$ , match the first  $n - m$  jobs, fail cheap  $r$  times at the next job, and mismatch all  $m$  cached completions. This attains its term using  $r + m$  writes. The analogous maximum-job witness attains  $\Omega + Ba_n$  at  $B \leq r$ .

*Lower bound for arbitrary programs.* Fix  $m \leq \min(B - r, n)$  and target the  $m$  heaviest jobs. Write a fresh own-key identity only at target comparisons whose zero-write continuation would match. Let  $f$  count these useful cheap failures,  $c$  useful cached mismatches and  $t$  all target completions. Stop forever after  $f = r + 1$  or  $t = m$ . At operation boundaries the write count is  $d = f + c$ , with  $c \leq t$ . Before another write,  $d + 1 \leq f + t + 1 \leq r + m$ . This is a globally  $(r + m)$ -bounded adversary even on inadmissible programs.

An admissible program cannot reach  $f = r + 1$ , and every target eventually completes. Before all targets finish, no target comparison can match: a useful comparison is invalidated, and a stale one already fails or protects. Thus all targets complete protectively and  $L \geq \Omega_m$ . Lemma 3, applied at the realized final count  $d$ , gives

$$\Omega_m \leq L \leq \Omega_{(d-r)_+}, \quad d = f + c \leq r + m.$$

Positivity forces  $d = r + m$ , hence  $f = r$  and  $c = m$ . All target completions are useful cached mismatches; this is a consequence of the contract, not an assumed normal form. Lemma 2 now charges  $r$  distinct failed preparations of work at least  $a_{n-m+1}$ , all  $m$  duplicate target kernels and mandatory  $\Omega$ . Maximizing over  $m$  gives the lower bound.

*Early budgets.* At  $d \leq r$ , zero protection forbids fresh or mismatching cached completion. The  $m = 1$  witness just constructed has  $r$  useful cheap failures at a maximum-work job, then its useful cached mismatch. Truncate after the completed call containing the  $B$ th useful write, for  $1 \leq B \leq r$ . Its unchanged prefix has destroyed  $B$  distinct paid preparations of that unfinished job. Eventual completion needs another distinct kernel, giving  $\Omega + Ba_n$  by global charging. At  $B = 0$  mandatory work suffices. Lower bounds include retained records, free inspections and paid early preparations after readiness.  $\square$

The maximizing  $m$  need not be largest:  $(1, 100)$  and  $r = 1$  give terms 200 and 102. The least  $W/L$  curves may have different maximizing executions. Other optimal- $L$  policies can use more work; relaxing protection changes the comparison.



*Dependency boundary.* The lower bound is order-independent. For any DAG, a topological cheap-then-cached policy attains  $F_r(B) = \Omega + Bw_{\max}$  at  $B \leq r + 1$ , retaining optimal  $L$  at all budgets. At  $r = 0$ , all-cached gives  $F_0(B) = \Omega + \Omega_B$ . For positive retries, Theorem 4 supplies a uniform work factor; a least entire DAG curve remains unresolved. The archive gives dependent-job observation examples. Section 7 summarizes the distinct supplied-budget observation boundary.

## 5 A Safety Filter That Needs No Future Job Data

A DAG scheduler must decide which protected computations preserve the contract. Figure 1 separates its job priorities, the safety test and the completing wrapper. The filter exposes every safe mode for the residual contract; the subsequent work theorem bounds caller freedom, including other cheap-phase orders. Additional fresh permissions alone need not improve the worst-work curve.

Fix the finite instance  $J$  and a completed ideal  $D$ , with  $S = J \setminus D$ . Let  $k$  count cached mismatches since leaving the cheap phase and  $\ell$  be protected work incurred. Each such mismatch completes a distinct job, so physical states satisfy  $0 \leq k \leq |D|$ . In this section, each remaining job completes once, either fresh or by *current capture and immediate preparation* followed by cached completion. No further cheap calls, early preparations, retained guards or partial kernels are allowed. A scheduler can use its available history to choose a ready job in finite time. It receives no future write bound.

*The residual contract.* A state is viable if there exists one such completing policy  $P$  satisfying

$$\forall b \geq 0 \forall E \in \mathcal{E}_b : \quad \ell + L_{\text{future}}(P, E) \leq \Omega_{k+b}(J). \quad (1)$$

Here  $\mathcal{E}_b$  concerns future writes and has an unconditional cap on every play. The same  $P$  must work for every  $b$ . This is an explicit interface obligation: it does not assert that  $k$  is the total number of past writes. A history exposing additional writes can support a weaker global obligation, so necessity below is local to (1).

**THEOREM 3 (EXACT RESIDUAL SAFETY).** *On every DAG, for  $0 \leq k \leq |D|$ , the following are equivalent:*

- (1) *some residual policy satisfies (1);*
- (2) *all-cached completion in any causal ready order satisfies it;*
- (3)  $\ell \leq \Omega_k(D)$ .

*From a viable state every ready cached action is safe. A ready fresh action  $i$  has a safe continuation exactly when*

$$\ell + w_i \leq \Omega_k(D \cup \{i\}). \quad (2)$$

*Thus this predicate, together with all cached actions, is maximally permissive for the stated residual contract and action class.*

**PROOF.** Specialize Theorem 5, proved in the next section, to  $p_i = w_i$  and  $h_i = 0$ . With physical  $k \leq |D|$  and positive completed weights, its region is  $\text{Top}_k(w_D \cup 0_S) = \Omega_k(D)$ . Every cached limit is the same sum, while each fresh limit is  $\Omega_k(D \cup \{i\}) - w_i$ . Thus all cached successors and exactly the displayed fresh successors are viable. Each completes one job, so selecting cached in any ready order preserves the condition through termination. The general theorem also proves necessity against every causal residual policy, including policies using concrete identities.  $\square$

*Composing the filter with retries.* Use any immediate-preparation cheap controller until all jobs finish or  $r$  cheap failures occur. At the switch initialize  $k = \ell = 0$  and retain *all* cheaply completed weights in  $D$ . The filter is initially viable. Cached match and mismatch preserve its invariant because  $\Omega_k(D \cup \{i\}) \geq \Omega_k(D)$  and  $\Omega_{k+1}(D \cup \{i\}) \geq \Omega_k(D) + w_i$ ; fresh uses (2). Every earlier cheap failure

**Caller** supplies each ready job and permitted mode. **Filter** keeps completed weights,  $k, \ell$ . **Wrapper** executes Table 2 and returns completion/outcome. The caller retains the DAG and saved outputs; the filter receives neither future weights nor  $B$ .

```

1   $f \leftarrow 0; D \leftarrow \emptyset; k \leftarrow 0; \ell \leftarrow 0.$ 
2  while unfinished jobs remain and  $f < r$ :
3     $i \leftarrow$  caller's ready job;  $\rho \leftarrow \text{Prepare}(i).$ 
4    if Cheap( $i, \rho$ ) succeeds: record completion and insert  $w_i$  into  $D$ 's heaps; else  $f \leftarrow f + 1.$ 
5  while unfinished jobs remain:
6     $i \leftarrow$  caller's ready job. Permit cached; permit fresh iff  $\ell + w_i \leq \Omega_k(D \cup \{i\}).$ 
7     $\text{mode} \leftarrow$  caller's choice among permitted modes.
8    if cached:  $\rho \leftarrow \text{Prepare}(i);$  execute Cached( $i, \rho$ ).
9    On mismatch:  $\ell \leftarrow \ell + w_i; k \leftarrow k + 1.$ 
10   else: execute Fresh( $i$ );  $\ell \leftarrow \ell + w_i.$ 
11   Record completion; insert  $w_i$  and restore the two-heap partition.
```

Fig. 1. The composed caller protocol defining  $\mathcal{A}_r$ . Each choice is causal and terminates. Preparation occurs exactly once before each cheap/cached call and never before fresh. The top-sum test uses completed weights and  $w_i$ , not undisclosed jobs.

and later cached mismatch needs a different write: their current-capture/comparison intervals are disjoint. Hence  $r + k \leq B$  whenever the switch occurs in a  $B$ -write execution. The invariant yields

$$Q \leq n + r, \quad L = \ell \leq \Omega_k(D) \leq \Omega_{(B-r)_+}(J).$$

If completion precedes the switch,  $L = 0$ . Theorem 1 shows that this protection curve is optimal even against the larger general-program class.

*The work cost of scheduling freedom.* Let  $\mathcal{A}_r$  contain exactly the composed callers just described: one current preparation per cheap attempt until  $r$  failures or completion, then only filtered fresh/cached completions, with one current preparation before cached and none before fresh. Ready-job and permitted-mode choices may depend on history, but not  $B$ ; each choice terminates. No redundant kernels or further cheap calls occur. Define  $F_J(B) = \inf_{P \in \mathcal{V}_r} W_P(B)$  over the full simultaneous-contract class of Section 4. A different policy may attain this infimum at different  $B$ .

**THEOREM 4 (SHARP UNIFORM WORK BOUND FOR EVERY CALLER).** *For every finite DAG, integer  $r \geq 0$ , caller  $A \in \mathcal{A}_r$  and budget  $B \geq 0$ ,*

$$W_A(B) \leq \alpha_r F_J(B), \quad \alpha_r = \frac{3r+2}{2r+2} = \frac{3}{2} - \frac{1}{2(r+1)}.$$

*Every caller is exact at  $B \leq r + 1$ ; for  $r = 0$ , every caller is exact at all budgets. For each fixed  $r \geq 1$ , no smaller constant works uniformly over all finite instances and all callers in  $\mathcal{A}_r$ .*

**PROOF.** The preceding composition proves  $\mathcal{A}_r \subseteq \mathcal{V}_r$ . On a complete caller execution, exact kernel accounting gives

$$W = \Omega + \sum_{\text{cheap failures}} w_i + \sum_{\text{cached mismatches}} w_i.$$

Failed-call preparations and mismatching cached completions need writes in distinct current-capture intervals. Put  $M = \max_i w_i$ . For  $B \leq r$  this gives  $W_A(B) \leq \Omega + BM$ . For  $B > r$ , put  $m = \min(B - r, n)$  and  $T = \Omega_m$ . If completion precedes the switch, there are no cached mismatches. Otherwise exactly  $r$  cheap failures precede the suffix, and at most  $m$  distinct suffix jobs mismatch. In either case  $W_A(B) \leq \Omega + rM + T$ , regardless of caller order or safe-mode choices.



Theorem 2's lower construction works on any DAG: its globally capped writer stays silent while predecessors complete. Admissibility then forces every target to complete, without assuming simultaneous readiness. Thus its full-program lower gives

$$F_J(B) \geq \Omega + \max_{1 \leq s \leq m} \{ra_{n-s+1} + \Omega_s\} \geq \Omega + \max\{(r+1)M, T\}.$$

The early-budget lower matches the caller upper. The same is true at  $B = r + 1$ , and at every  $B$  for  $r = 0$ .

For  $r \geq 1$ , normalize  $x = T/M \geq 1$  and  $z = \Omega/M \geq x$ . The ratio is at most  $(z+r+x)/(z+\max(r+1, x))$ , nonincreasing in  $z$ . Set  $z = x$ . For  $x \leq r+1$  the resulting  $(2x+r)/(x+r+1)$  increases; for  $x \geq r+1$ ,  $1+r/(2x)$  decreases. Both meet at  $\alpha_r$ . The execution upper and each full-program lower hold separately, so no interchange of infimum and supremum is needed.

For sharpness, fix  $r \geq 1$ , set  $q = r/(r+1)$ , and use independent descending weights  $w_j = q^{j-1}$ . Use descending cheap order, retrying the current unfinished job until it completes or the  $r$ th global failure occurs; after that failure, immediately prepare/cache each remaining job. At  $B = n+r$ ,  $r$  first-job failures followed by  $n$  mismatches give its exact worst work  $2\Omega + r$ . Every such action is permitted. Since  $rw_s + \Omega_s = r+1$  for every  $s$ , Theorem 2 gives  $F_J(B) = \Omega + r + 1$ , where  $\Omega = (r+1)(1-q^n)$ . Their ratio tends to  $\alpha_r$ . Multiplying by  $(r+1)^{n-1}$  yields positive integer weights with the same ratio. Every smaller constant therefore fails on a finite input.  $\square$

For one spare call,  $\alpha_1 = 5/4$ : a scheduler may pursue other ready-job priorities while preserving optimal protection and a 25% bound on worst body-work overhead. The same filter need not receive future weights or edges. Sharpness concerns the entire caller class; minimum-ready is ascending and optimal on the independent geometric witnesses. The theorem neither solves the exact positive-retry DAG curve nor bounds native waiting, allocation, writer costs or filter maintenance.

*What the filter stores.* Maintain the completed weights in two heaps: the largest  $k$  in a min-heap, the rest in a max-heap, together with the top- $k$  sum  $T$ . For  $k = 0$ , positive fresh work is forbidden. Otherwise let  $\tau$  be the minimum of the first heap; the fresh test is

$$\ell + w_i \leq T + \max\{0, w_i - \tau\}.$$

It takes  $O(1)$  arithmetic/comparison operations. Completion inserts one weight, and a mismatch increases  $k$  by one; a constant number of heap transfers gives  $O(\log(|D| + 1))$  update time and  $O(|D|)$  storage. Starting empty needs no preprocessing. Integer bit costs are additional.

No future weight or edge enters this predicate. The scheduler checks readiness and proposes a job/mode; the safety component returns the permitted modes and updates only after completion. Thus the same component works when future jobs are undisclosed, provided each realized instance is finite and obeys the interface. This supplies an invariant for each finite instance; infinitely arriving jobs have no completion guarantee.

*Permissiveness and total work.* The distinction is observable even on a chain. Let  $r = 1$ : cheaply complete weight 100, fail once at weight 1, then mismatch its cached completion. Now  $k = \ell = 1$ ; a fresh weight 2 is safe because  $1 + 2 \leq 100$ . A rule that waits until *all* remaining work fits the current allowance would reject it when the remaining weights are  $(2, 80, 80, 80)$ . The smaller interface permits this decision without knowing those later weights.

Safe choices need not minimize  $W$ . However, any immediate fresh/cached suffix has worst total work no greater than all-cached after the *same cheap controller*. For a concrete execution retain its exact prefix with  $b_0$  actual writes and duplicate cheap work  $C$ . If  $m$  suffix jobs mismatch, its work is at most  $\Omega + C + \Omega_m(S)$ . An all-cached continuation can attain this upper expression by targeting the  $m$  heaviest remaining jobs, using  $b_0 + m \leq B$  writes. Script that prefix and cap the

resulting writer unconditionally at  $b_0 + m$ . This proves worst-curve domination, not dominance under the same write timing. Therefore ascending cheap order retains Theorem 2's least curve for independent jobs under *any* filtered suffix; at  $r = 0$  the same holds on every DAG. Theorem 4 bounds the cost of other cheap-phase priorities; exact positive-retry DAG work optimality remains open.

## 6 Exact Safety with Mismatch Premiums

Keep the residual actions and write semantics, but charge fresh completion  $p_i > 0$ , cached match 0, and cached mismatch  $q_i = p_i + h_i$  with  $h_i \geq 0$ . These are fixed supplied protection costs. For a multiset  $A$ , let  $\text{Top}_j(A)$  sum its largest  $\min(j, |A|)$  entries, with  $\text{Top}_0 = 0$ . Put  $G_j = \text{Top}_j(q_j)$ . The benchmark is the *all-cached charged curve*, not an asserted optimal charged curve. One completing policy, without  $b$  as input, must satisfy  $\ell + L_{\text{future}} \leq G_{k+b}$  for every integer  $b \geq 0$  and every unconditionally  $b$ -bounded future writer.

**THEOREM 5 (EXACT CHARGED RESIDUAL SAFETY).** *For every finite DAG and physical  $0 \leq k \leq |D|$ , the exact viable region is*

$$\ell \leq V(D, k) := \text{Top}_k(q_D \cup h_S). \quad (3)$$

*For each ready  $i$ , set  $X_i = q_D \cup h_{S \setminus \{i\}}$ . Its exact permissions are*

$$\text{cached } i : \quad \ell \leq \text{Top}_k(X_i), \quad (4)$$

$$\text{fresh } i : \quad \ell + p_i \leq \text{Top}_k(X_i \cup \{q_i\}). \quad (5)$$

*Every ready job has at least one safe mode at a viable state. Future edges and future body costs are absent from these predicates; future premiums can matter.*

**PROOF.** *One policy for all caps.* Fix a causal deterministic policy and any admissible prefix. Restrict writers to silence or one fresh-identity write at each cached gate, and fix a rule selecting the concrete fresh identities. Fixing that rule and the deterministic policy fixes every concrete history at a match/mismatch node. Its unfolding is binary with depth at most  $|S|$ , since each action finishes a job; identity-dependent choices therefore also induce a finite tree. A leaf with  $c$  mismatches and future cost  $t$  has a causal script writer. Cap that writer at  $c$  writes on *every* play. It reproduces the leaf, since none of its first  $c$  writes is suppressed and its selected play has no later write. Thus safety requires  $\ell + t \leq G_{k+c}$  at each leaf. The cap is never supplied to the policy.

Conversely, a tree satisfying every leaf inequality gives a policy that uses only abstract outcomes. Any actual  $b$ -write execution follows a leaf with  $c \leq b$  mismatches, by the disjoint comparison intervals. Extra harmless writes only enlarge  $G_{k+b}$ . This proves an exact completion-tree characterization for the residual contract. It does not require identical choices at different concrete histories with the same numerical state, nor characterize a weaker global obligation after observations revealing extra writes.

For a tree  $P$ , its exact permissible prefix cost is  $\min_{\text{leaf}}(G_{k+c} - t)$ . Define  $V$  as the maximum of these limits over the finitely many abstract completing trees. At termination  $V(J, k) = G_k$ . Fresh  $i$  has threshold  $V(D + i, k) - p_i$ , and cached  $i$  has threshold

$$\min\{V(D + i, k), V(D + i, k + 1) - q_i\}.$$

The two outcome subtrees may differ, but both belong to the same policy and satisfy all suffix caps. Take the maximum over modes and ready jobs. This is backward induction over jobs, not separate optimization at each budget.

For the local algebra, write  $C_j = \text{Top}_j(X)$  and  $Y_j = \text{Top}_j(X \cup \{q\})$ , with  $q = p + h$ . Then

$$\min\{Y_k, Y_{k+1} - q\} = C_k, \quad \max\{Y_k - p, C_k\} = \text{Top}_k(X \cup \{h\}).$$

For  $1 \leq k \leq |X|$ , let  $\tau$  be its  $k$ th entry. If  $q \leq \tau$ ,  $Y_k = C_k$  and  $Y_{k+1} \geq C_k + q$ ; otherwise  $Y_{k+1} = C_k + q$ . This proves the first identity. Since  $Y_k = C_k + \max(0, q - \tau)$ , the second has increment  $\max(0, h - \tau)$ . At  $k = 0$  both identities are zero; for  $k > |X|$  they reduce to saturated sums. Ties are included.

Inductively, both child multisets for ready  $i$  are  $X_i \cup \{q_i\}$ . The identities give the two displayed mode thresholds and their maximum  $\text{Top}_k(q_D \cup h_S)$ , independently of  $i$ . The terminal multiset is  $q_J$ , completing the induction. Choosing a safe mode for any ready job preserves viability and terminates after  $|S|$  calls.  $\square$

*Fixed assignments already attain feasibility.* For a fixed set  $F \subseteq S$  of fresh jobs, with all others cached in any ready order, the exact prefix allowance is  $\text{Top}_k(q_D \cup q_F) - \sum_{i \in F} p_i$ . To see this, minimize  $G_{k+|M|} - \sum_{i \in M} q_i$  over cached mismatch subsets  $M$ : the minimum is the top- $k$  sum on  $D \cup F$ , attained by choosing cached entries above its  $k$ th entry (empty at  $k = 0$ , saturated when necessary). Maximizing over  $F$  selects only fresh entries included in that top sum, replacing each selected  $q_i$  by  $q_i - p_i = h_i$ . This gives Equation (3); forcing inclusion or exclusion of a ready  $i$  gives its two mode limits. Thus outcome adaptation does not raise this feasibility threshold. The filter exposes all safe choices; its work guarantee is the separate Theorem 4.

*When completed information suffices.* All-cached has threshold  $C = \text{Top}_k(q_D)$ : minimize  $G_{k+m} - \text{Top}_m(q_S)$  over  $m$ . The top completed/future subsets give the lower bound; taking future entries above the  $k$ th completed entry gives equality (use  $m = 0$  at  $k = 0$ ). For  $k > 0$ , Equation (3) equals  $C$  exactly when no future premium exceeds that completed entry. Pointwise equality need not persist after a mismatch. A sufficient global contract is  $\max_J h_i \leq \min_J q_i$ . Common  $h$  satisfies it; then adding  $hk$  to both incurred body protection and its top- $k$  allowance cancels, recovering the body-only invariant and fresh test.

*The general interface.* Store  $A = q_D \cup h_S$ , its top- $k$  sum  $T$ , and adjacent order statistics  $\tau = A_{(k)}$ ,  $\sigma = A_{(k+1)}$ , padding missing entries by zero. Exact query thresholds are

$$C_i = T - \max(0, h_i - \sigma), \quad F_i = T - \min\{p_i, \max(0, \tau - h_i)\},$$

with  $F_i = -p_i$  when  $k = 0$ . Deleting  $h_i$  replaces it by  $\sigma$  only when it lies in the top  $k$ ; replacing it by  $q_i$  gives the fresh expression. Two indexed heaps provide  $O(1)$  queries and  $O(\log n)$  completion updates: replace  $h_i$  by  $q_i$  and increment  $k$  only on mismatch. This version receives all future premiums, uses  $O(n)$  storage and initializes in  $O(n + k \log(n + 1))$  operations. The scheduler still establishes readiness; the safety component receives no future edges or future body costs.

**PROPOSITION 6 (FUTURE FEES CAN CHANGE A PRESENT PERMISSION).** *Over unrestricted fees, the visible prefix and completed/current costs do not determine the exact safe-mode set, even with the entire DAG and vector  $q_J$  supplied.*

**PROOF.** Use the chain  $0 \rightarrow 1 \rightarrow 2$ , with  $q = (2, 2, 4)$  and  $p_0 = p_1 = h_0 = h_1 = 1$ . Future job2 is  $(p_2, h_2) = (1, 3)$  in A and  $(3, 1)$  in B. Both curves are  $(0, 4, 6, 8)$ . Initially cache0 and observe a mismatch after one write:  $D = \{0\}$ ,  $k = 1$ ,  $\ell = 2$ . This prefix is reachable under the safe all-cached policy in both worlds. For proposed fresh1, Equation (5) gives limit  $\max(2, h_2, 2) - 1$ , namely2 in A and1 in B. Fresh1 is therefore permitted only in A despite identical stated filter inputs. Theorem 5 supplies a single all-budget continuation in A and excludes every such continuation in B.  $\square$

The completed-only filter rejects fresh 1 in both worlds; the general filter admits it in A. After A's fresh 1,  $q_D = \{2, 2\}$ ,  $k = 1$ ,  $\ell = 3$ , and future  $h_2 = 3$ . This is viable, but cached 2 is forbidden: one mismatch costs7 above  $G_2 = 6$ . Thus permitting every cached action throughout the larger region would be incorrect. More permissions establish neither lower worst total work nor native latency gains.

*Charging all three outcomes.* The same completion-tree argument handles fixed actual prices: cached match  $a_i \geq 0$ , fresh  $f_i \geq 0$ , and cached mismatch  $b_i \geq \max(a_i, f_i)$ . To avoid confusing  $b_i$  with a write cap, denote the latter by  $u$ . Put  $A(U) = \sum_{i \in U} a_i$ ,  $q_i = b_i - a_i$ ,  $p_i = f_i - a_i$ ,  $d_i = (-p_i)_+$  and  $t_i = b_i - \max(a_i, f_i)$ . The actual all-cached benchmark is  $A(J) + \text{Top}_{k+u}(q_J)$ .

**THEOREM 7 (THREE FIXED ACTUAL PRICES).** *For actual incurred protection  $\lambda \geq 0$ , define  $\ell = \lambda - A(D)$ , which may be negative. One completing policy satisfies the actual benchmark for every future cap  $u$  exactly when*

$$\ell \leq \sum_{j \in S} d_j + \text{Top}_k(q_D \cup t_S). \quad (6)$$

*For ready  $i$ , let  $R_i = \sum_{j \in S \setminus \{i\}} d_j$  and  $X_i = q_D \cup t_{S \setminus \{i\}}$ . Add  $R_i$  to the cached/fresh limits of Theorem 5, using this  $X_i$  and signed  $p_i$ . Every ready job has a safe mode.*

**PROOF.** Subtract the once-per-job baseline  $A(J)$  from every complete leaf and benchmark. Increments become 0,  $p_i$ ,  $q_i$ , with current  $\ell$ , so the preceding tree recurrence applies. For  $p_i \geq 0$ , its maximum identity gives premium  $q_i - p_i = t_i$ , including zero. For  $p_i < 0$ , fresh dominates and gives  $-p_i + \text{Top}_k(X_i \cup \{q_i\})$ , with  $d_i = -p_i$ ,  $t_i = q_i$ . Adding the other future savings  $R_i$  proves the region and mode limits by induction. Nonnegative  $q_i$  keeps the cached minimum identity valid; ties, zeros and saturation are included.  $\square$

For fixed guard, comparison and body prices  $g_i, c_i, w_i$ , set  $(a_i, f_i, b_i) = (g_i + c_i, g_i + w_i, g_i + c_i + w_i)$ . Then  $q_i = w_i$ ,  $d_i = (c_i - w_i)_+$ ,  $t_i = \min(c_i, w_i)$ , with every guard/comparison charge retained. If  $c_i > w_i$ , fresh is cheaper than match and signed  $\ell$  records savings. This formula requires future  $d_i, t_i$  and is outside the positive- $p$  heap implementation. Supplied prices exclude waiting, partial work and variable native costs.

## 7 Observation and Call Accounting Boundaries

*Observing cached comparisons.* The filter needs the cached comparison outcome, including when kernels return constant objects. Erasing it requires erasing the entire comparison-dependent visible transition: no compared input, Boolean, selected-record change or callback-local effect survives. Earlier snapshots [23] prove the consequence for independent works (8, 16), supplied  $B = 1$ ,  $Q \leq 2$ : the visible  $(W, L)$  frontier is  $\{(24, 24), (32, 16)\}$ , but strict erasure changes it to  $\{(24, 24), (40, 16)\}$ . The archive retains the full live/saved/record coupling and dependent extensions. This is preceding boundary evidence, not a new theorem or an assumption used to suppress observations in the present model.

*Counting native attempts.* Let  $c$  count cached mismatches. Charging them twice gives  $Q' = Q + c$ . At completion,  $Q' = n + z$  for cheap failures plus cached mismatches  $z$ . Earlier analysis [23] shows that the universal constraint  $Q' \leq n + r$  changes the least protected curve to zero for  $B < r$  and  $\Omega$  for  $B \geq r$ . Thus the call unit is part of the contract. The Linux counts below retain this distinction.

Maldonado et al.'s irrevocable transition validates an executed prefix, retaining it on success and restarting irrevocably on failure [44, Section III.D]. For a whole kernel, grouping the failed transaction and restart as one episode gives  $Q$ ; counting both attempts gives  $Q'$ . Their partial prefixes can protect only  $w - u$  after  $u$  valid units execute outside protection. Our whole-kernel lower bounds do not cover that larger action language.

## 8 Conditional Transfer to Java Map Operations

Java 17's per-key map interface provides a concrete interpretation [18–20]. The result below is conditional on specified API/source properties and concerns completed traces. It is not a proof of arbitrary Java clients or the JDK/JMM.

*Premises.* Retain Section 2’s controller, kernel, saved-output and termination requirements. Map keys are permanent and distinct, with stable, pure, terminating hashing/equality/ordering. Values are non-null, deeply immutable and reference-equal; replacement may call equals. Writers preserve membership, value shape and completion milestones, and admit references fresh to all prior target records/snapshots. Kernels make no recursive or other-key map updates. Native waiting, allocation and traversal are separate from body work.

Use three per-key synchronization premises. A successful retrieval returns a value live at a point within its call. Conditional replacement checks/stores atomically. A normally completing compute invokes its remapping once and excludes same-key publications through its store. The pinned source’s list and tree paths remap/store at lines 1940–1942 and 1975–1978, returning the saved value at 2003; replacement saves success before returning at 1174. Forwarding and structure-only node copies preserve stored value references. These are the API/source assumptions used, rather than consequences of the finite tests.

Cheap wraps replace or validation-only compute and returns only success/failure. Fresh always computes; cached returns its saved output and comparison Boolean. The controller may retain payloads, reference equality, records, saved parents and selector state. Raw cheap-mismatch values, numeric allocation IDs, clocks and diagnostics are not observations. Erasure additionally needs Section 7’s complete restriction.

*The relation at completed-operation boundaries.* Let  $H$  be the concrete heap. An injective map  $\eta$  carries value references to abstract identities, preserving aliases and immutable payloads. Let  $\Delta$  be a proof-only ordered queue of committed external assignments deferred during normalization. Relate  $H$  to  $\sigma = (x, \mathcal{M}, \mathcal{R}, \lambda)$  by the following conditions:

- (1)  $\text{decode}_\eta(H.\text{map}) = \text{apply}(\Delta, x)$  and  $b_H = b_x + |\Delta|$ , counting committed external publications.
- (2)  $\mathcal{M}(i) = \eta(\text{done}[i])$  is the saved completion output. A later map write never changes it.
- (3) Each selectable record corresponds to an actually available handle  $(i, \eta(s), \mathcal{M}|_{\text{Pred}(i)}, \eta(z))$ ; a discarded ghost computation supplies no selectable record. Observable locals and reference relations agree.
- (4) Completed wrapper calls and grouped kernel invocations have the same  $Q, W, L$ . A full kernel charge and its output are introduced only after the source kernel returns.

The queue contains committed assignments, not callbacks to be executed again. Its presence handles publication followed by a writer before the wrapper returns: the wrapper still saves its own result even though the live entry has changed.

**THEOREM 8 (CONDITIONAL RESOURCE TRANSFER FOR ACTUAL WRITES).** *Under these premises, the body-work bounds and lower witnesses of Sections 3, 4 and 5 hold for actual external writes, including colliding bins. Completed traces preserve declared observations,  $Q, W, L$ , operation kinds and cached mismatches, hence also  $Q'$  of Section 7.*

**PROOF.** *Projection for upper bounds.* For a completed unprotected kernel, retain its captured operands and group its actual invocation, returned record and full charge into Prepare. Defer writes between capture and return in  $\Delta$ . This maintains the relation after the source body returns; it never assigns a completed result to an unfinished prefix. For each next operation, flush the pending assignments through its selected atomic/retrieval point and retain later commits. This constructs one consistent projection; a unique normalization is unnecessary. For an inspection, flush commits through its retrieval point, return the saved reference, and defer later commits.

During fresh/cached remapping, the own-key value is fixed through publication. Saved parents and all other kernel operands are immutable. Other-key commits may therefore precede the corresponding abstract effect without changing its result or readiness: completion milestones and



saved operands are immutable. This commutation concerns observed values and body charges; bin contention and waiting are excluded. The protected body and publication give precisely Table 2's update and charge. Extend  $\eta$  on new output references. After publication, further writes enter  $\Delta$ ; on return, save the prepared output for successful cheap or the callback's returned output, never a map reread. Thus the saved-output and live-map clauses both hold. Structure-only changes preserve value references; a concurrent value publication remains a write.

Finally flush any terminal queue as a zero-foreground-cost write suffix. The resulting completed trace preserves observations and costs without adding writes. Our explicit upper policies use boundedly many finite kernel/call segments; selected-operation progress therefore supplies termination, and the abstract inequalities transfer. Completed-trace projection alone would not prove arbitrary low-level termination or equivalence of writer diagnostics.

*Realizable lower witnesses.* Suspend the controller after its job/mode and prior store are fixed but before API entry. No foreground guard is held. Run a terminating own-key writer that installs a fresh legal reference, then resume the call. This works even when bins collide, and invalidates every prior target preparation. Repeating the restricted target adversaries gives the lower proofs' comparison outcomes and distinct kernel charges, including early computation. A concrete script with a physical cap of  $d$  writes belongs to  $\mathcal{E}_d$  on every play and preserves its designated  $d$ -write trace. It suffices to realize these witnesses; we need not implement every abstract environment. Immutable saved parents preserve readiness, and the Boolean supplies the permitted branches.

Operation and mismatch labels are preserved by both constructions, establishing the stated  $Q'$  accounting as well. This argument uses API/source synchronization premises; it does not derive them from trace agreement.  $\square$

Earlier fixtures execute  $8w_i$  kernel iterations; newer wrappers execute  $w_i$ . Neither scale charges native allocation, waiting, writer work or map internals. Full source control points and the relation are archived. Finite tests do not establish concurrent-resize or JMM correctness.

## 9 Formal Corroboration and Operational Evaluation

Executions use Apple M 2/24 GiB, Python 3.10.9/3.12.14 and OpenJDK 17.0.19; the pinned map source was inspected, not rebuilt. Hashes bind code, inputs and traces. Codex drafted proofs, implementations, input generators, checker/analysis scripts, figures and artifact packaging, and ran experiments. Claude supplied technical critiques and fixed-manuscript reviews. These are author-side AI-assisted checks; no human study or independent certification is claimed.

Inputs fix graph/works/prices; roots add budget/retries. Paths are complete branches; executions are native invocations. Pairs compare policies; coordinates add budgets. Studies overlap.

### 9.1 RQ1: Do the Formulas and Implementations Agree?

Table 3 compares separate game/tree evaluators with formulas and heaps with sorting. Retry roots add  $r$ , budget rows add  $B$ ; residual states need not be initially reachable. Populations overlap and corroborate mathematics; they are not sampled software clients.

For the all-caller test, the explorer maximizes over every ready job, permitted mode and feasible outcome, with  $r = 0, 1, 2$  and  $B = 0, \dots, n + r$ . The independent ascending reference equals Theorem 2's curve. The repeated 96,795 count in the first two rows belongs to different studies; neither replaces the full-program lower argument.

For the policy comparison, both use minimum-ready cheap, then maximum-ready cached before a mismatch; afterwards they use maximum-ready safe fresh, otherwise minimum-ready cached. More permissions give no worst-work improvement here. A separate local crossing of suffix curves also disappears in complete initial curves. The positive-retry DAG exact optimum remains unresolved.



Table 3. Finite corroboration of the core results. Zero discrepancies concern the stated finite scopes, not arbitrary-program proof certification. Exact inputs, every output and earlier corrected attempts are archived.

| Question                                   | Fixed coverage                                                                                                             | Result and boundary                                                                                                                                            |
|--------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Retained records / exact independent curve | 96,795 retained-record roots on all forward DAGs, $n \leq 4$ , works $\{1, 2, 4\}$ ; separate 647 independent roots.       | Immediate/retained frontiers agree; independent curve agrees. The finite game omits dominated moves.                                                           |
| All-caller work factor                     | Same 5,421 graph/weight inputs; 16,263 retry roots, 96,795 budget rows, 4,632,516 states; 72 integer geometric examples.   | Every $Q/L/W$ bound holds; early/ $r = 0$ exactness and sharpness arithmetic agree. Full general programs are not enumerated.                                  |
| Body residual formula                      | 493 roots, 95,514 physical and 62,949 surplus-credit states; $n \leq 3$ , works $\{1, 3, 7\}$ , plus four $n = 4$ vectors. | Zero disagreement; 936 fresh permissions beyond the earlier rule. Initial reachability is not required.                                                        |
| Body heaps                                 | 8,355 initial states, 58,485 queries, 166,372 actions and 48,000 stream steps.                                             | All match sorting; 9,083 unsafe fresh requests rejected. Bit costs/allocation are separate.                                                                    |
| Policy work effect                         | 16,267 roots, 1,565,245 interpreted paths; all forward DAGs up to four jobs, works $\{1, 3, 7\}$ , plus four diagnostics.  | Both filters' worst-work curves are equal. A predicted worsening was falsified and retained.                                                                   |
| General fee formula                        | Earlier 379,647 physical states; new 130,832 games/2,533,120 states; separate 12,288 full-tree cases.                      | Zero discrepancy. Full-tree benchmark is computed from mismatch subsets; future-fee separations remain.                                                        |
| General fee heaps                          | 42,260 initial states, 192,540 query pairs, 577,620 actions; 39 streams up to 4096 jobs.                                   | Zero discrepancy; 20,478 distinct updates. Original timeout retained, as explained below.                                                                      |
| Three actual prices                        | 157,465 state games on every labeled DAG up to three jobs and eight price triples; 1,934,433 policies/7,964,521 leaves.    | All 247,312 mode limits agree; no time-out or omitted input. Includes zero and negative normalized costs; positive- $p$ heaps do not implement this extension. |

The fee-heap stream check originally timed out after 16,972 updates. A follow-up changed only state-copy verification and finished the two unfinished series; all 2,638 overlapping records agree and are counted once. Query code and scientific inputs were unchanged. All full-tree fee checks enumerate actual leaf costs; the three-price checker includes fresh cheaper than cached match. Original larger fee witnesses and all adverse/corrected attempts remain available. None of these agreements establishes native cost transfer.

*Java implementation of the residual interface.* The Java 17 component receives completed weights,  $k, \ell$ , current weight and public outcome. Linked binary heaps provide the abstract bounds; long overflow rejects, and allocation/GC time is unbounded. All 498 operation sequences and two regressions match sorting. Per-key compute runs 996 paths on 48 roots: all 31,892 events/outputs agree, including the chain's strict permission, 118 postpublication-write and 118 double-write paths. All 62 worst- $W$  coordinates equal the same cheap phase then all-cached. All-fresh violates zero-write protection. These authored fixtures are neither clients nor an all-interleaving proof.

Work excludes filter maintenance, allocation, parent capture, map internals, writers and waiting; latency benefit is unestablished.

Earlier Java bin-layout/allocation, corruption, stronger-observation and changed-call studies remain archived; they overlap these checks.

## 9.2 RQ2: A Native Traversal and Its Guard Boundary

Linux v6.12 `dentry_path_raw` walks mutable parents/names under RCU, validates `rename_lock`, then rebuilds under its reader-exclusive lock on failure [7]. Lock contention motivated this fallback [9]; the caller explicitly limits optimistic attempts. Our KUnit-only variant prepares a second traversal with a *new even* sequence, locks, then reuses on match or resets/rebuilds on mismatch. Components must be fully initialized before parallel reading; subsequent relevant changes must use the global sequence without wrap. This covers initialized `d_exchange` fixtures. It excludes concurrent initialization, including unsequenced `d_mark_tmpfile` name changes [8], mount traversal and `getcwd`.

The source bound assumes a private buffer  $1 \leq K \leq \text{INT\_MAX}$ , nonnegative int name lengths, valid `dentry`/RCU lifetimes and successful stable-name copies. With  $C = K - 1$ , counted names, copies, fault fills and slashes obeys  $T \leq 2C + 2$  per traversal; the archive gives the byte-level derivation. For validation failures  $X, Y$ , fresh capture gives  $X + XY \leq \min(B, 2)$  for actual global writer sections, including unrelated renames. Cached therefore has  $T_{\text{total}} \leq (1 + \min(B, 2))(2C + 2)$ ,  $L_T = 0$  for  $B \leq 1$ , and  $Q \leq 2$  counting lockless validation or guarded completion blocks. Guard/RCU instructions, waiting and postambles are separate; this is a single-job upper refinement, not the publication/observation contract.

Two-CPU ARM64 QEMU checks lockdep and atomic sleep. The gated writer uses `lock_rename` and `d_exchange`. All 1,920 rows (948 paths, 972 expected overflows) match quiescent-upstream strings, offsets, canaries and persistent outputs, without lock/RCU diagnostics; sequence brackets confirm writes. Four faulty variants reject: unvalidated overflow, unrestored buffer and omitted final validation corrupt outputs; old-sequence reuse violates protection at  $B = 1$ . Copy-fault/allocation failure and native-time performance remain untested.

Table 4. Source counts at scripted gates: `/driver/cache/entry/leaf` moves to `/alternate`, then back.  $G$  counts reader-exclusive acquisitions.

| Policy                 | One write |       |     |     | Two writes |       |     |     |
|------------------------|-----------|-------|-----|-----|------------|-------|-----|-----|
|                        | $T$       | $L_T$ | $Q$ | $G$ | $T$        | $L_T$ | $Q$ | $G$ |
| Upstream               | 41        | 12    | 2   | 1   | 58         | 29    | 2   | 1   |
| Cached                 | 41        | 0     | 2   | 1   | 70         | 29    | 2   | 1   |
| Two cheap, then locked | 41        | 0     | 2   | 0   | 70         | 29    | 3   | 1   |

A body attempt restarts at the original `dentry`/buffer, including overflow. Upstream has  $1 + X \leq 2$  attempts; cached has  $1 + X + XY \leq 3$ , relaxing that source bound while preserving completion under terminating primitives. All 384 cached/two-cheap pairs agree on output/body vectors: 138 first-invalid/second-valid pairs avoid the guard; 144 two-failure pairs need another logical block. Both make one public invocation. Counting RCU, sequence and exclusive-lock operations gives  $4R + 2G$  for  $R$  outside traversals. Cached/two-cheap counts are 4/4, 10/8, 10/10 for zero/one/two failures: equal maxima despite logical caps 2/3. No caller-required  $Q = 2$  SLA was found.

A stronger comparator snapshots names under protection and materializes outside. Its allocation, locks, capture and refcounts lie outside  $L_T$ ; five 64-slot overflows retain discarded capture work before fallback. Zero protected body work is therefore not zero protected work or a lower bound

for larger workspaces. The earlier 1,536-row matrix overlaps these populations. No timing benefit follows.

### 9.3 RQ3: Preservation and Cost Limits in Roslyn

Roslyn’s Workspace.cs at 6c4a46a3 transforms an immutable solution outside a semaphore and validates identity inside [10, 11]. Our SourceText-only patch switches after  $r$  shared failures to fresh (two-mode) or cached completion (three-mode); stronger guarded rebase still needs a protected merge. Lazy work, allocation, versions and cache-dependent merges prevent fixed-work transfer. No upstream numerical cap or SLA was found.

All 236 authored cases agree; 12 corruptions reject. In 96 compiler processes, 2,556 project projections match declared reference/parse-option/linked-text expectations under cold/warm caches; eight corruptions reject. Archived failures include a same-text deadlock and four missing-document timeouts; corrected patches pass 14 error cases and reentry controls. Both sides use Roslyn. Custom loaders, changed-text reentry and general concurrent equivalence remain unchecked.

Arrival tests compare shared/per-update caps, two/three modes and both baselines: eight balanced blocks, each serving 32 updates and 64 FIFO writers at 0, 10, 100, 1000  $\mu$ s. All 2,560 measured and 512 warm-up units pass source/count checks; 12 corruptions reject. At  $r = 2$ , local/shared caps permit 96/34 calls, precluding equal-cap comparison.

Shared three/two foreground-time ratios are 1.72 [1.40, 2.11] and 1.58 [1.17, 2.16] at 0/10  $\mu$ s, favoring two modes. Slower-arrival, writer-response and baseline ratios (0.83–1.08) include 1. Geometric means use eight paired blocks and descriptive 95% intervals from 10,000 fixed block-bootstrap samples, without multiplicity correction. Instrumentation overhead is unidentified; authored arrivals, eight blocks and one host limit inference.

## 10 Related Work and Research Lineage

*The contract and existing mechanisms.* Maldonado et al. [44, Section III.D] retain a valid transaction prefix on becoming irrevocable, or restart irrevocably after invalidation. Our cached operation uses that established protection/reuse idea; the new object is its resource guarantee under a whole-kernel completion-call contract. Their partial-prefix migration and native transaction attempts do not obey that contract automatically. Section 7 makes the accounting difference explicit.

*Fallback and contention.* Protected reexecution [6, Section 3.3], bounded lock-elision/RPC retries [26, 27, 31, 32], irrevocable prefix preservation [33], contention management [4, 24, 25] and switching/tuning [5, 41, 42] are established. HTM aborts need not consume writes [28], and RPC can fail. Their progress/time objectives differ from persistent-job  $W/L$  caps.

*Synthesis.* Černý et al. [15] synthesize retries, guards and caches for weighted limit-average safety (PSPACE by erratum); later work optimizes protection/profile costs [16] or preemption-safe locks [17]. We inherit global-budget/information-state minimax DP [12, 21, 29], backward induction and Pareto composition; our lower bounds supply an exact independent-job work curve, and our residual theorem replaces game enumeration for one explicit all-budget protection contract.

*Scheduling with testing.* At  $r = 0$ , fresh is untested execution with  $u_i = w_i$ ; cached tests for  $t_i = w_i$ , then executes for  $p_i \in \{0, w_i\}$ . Thus  $W$  counts testing plus execution;  $L$  counts execution. For any deterministic tree, hidden bad-job labels represent its mismatch set and own-gate writes realize them. Dynamic writing alone does not distinguish this canonical game.

Dürr et al. [35] introduce adversarial testing; Buld and Schulz [38] and Albers and Eckl [39] allow unequal test times/upper bounds under competitive time objectives. Damerius et al. [36] separate testing and execution with a *policy-side* budget; ours bounds adverse outcomes and permits

preparation at  $B = 0$ . Dufossé et al. [37] study minimax testing, revealing durations even for untested jobs; polynomial adaptive optimality assumes their two-phase conjecture. Albers and Janke [40] bound failing jobs for irrevocable parallel-machine assignment.

Our absolute  $W/L$  caps include retry slack. For the 8/16 adaptive and cache-both policies,  $(\sup T, \sup L) = (24, 16)$ , where  $T$  counts preparation, yet  $\sup(T + L) = 32$  and 40. The distinction is the absolute joint contract and its simultaneous all-budget quantifier. Adaptive testing and binary policy trees are inherited; dynamically placing a write is not by itself a novelty claim.

*Budgeted sums and safety filters.* Bertsimas and Sim’s cardinality-budget protection function [14, Section 3] supplies the top-weight sum; our sorted-union identity is elementary. Bloem et al. [45, Sections 3–5] enforce winning successors. Feng et al. [46, Sections 4.4–4.5] enlarge control envelopes through parameter inference; Cano et al. [47, Appendix A] build maximally permissive shields with history. These frameworks can encode our game. Our closed-form region specializes them: body-only costs discard future job data; arbitrary mismatch premiums require future fees, still no future body values or edges. Three actual prices add future savings. Fixed mode assignments already attain residual feasibility, so safety needs no feedback advantage. The all-caller theorem further bounds body work against the full simultaneous-contract optimum; the region alone supplies no such coefficient. Mismatches imply a write lower bound, not minimum history for general shields.

*Scheduling freedom.* Graham’s arbitrary-list bound [49, Theorem 1] and ter Mors et al.’s price of autonomy [50, Theorem 1, Proposition 4] already quantify performance under scheduling freedom. Graham’s  $1 + (m - 1)/m'$  even equals  $\alpha_r$  at  $m = r + 1, m' = 2(r + 1)$ , but those parameters count processors and its objective is makespan. Our comparison charges repeated kernels and gives both caller and full causal comparator the same universal  $Q/L$  contract, without extra processors or speed. Numerical coincidence alone supplies no reduction.

*Precedence and reuse.* The supplementary charged frontiers use established precedence [34] and scheduling-hardness constructions [1]; they do not prove general DAG work optimality for the present filter. Universal restart [43] concerns expected Las Vegas time. Build and incremental systems [2, 3, 30] may reuse partial work outside our whole-kernel interface.

## 11 Limitations and Threats to Validity

Our filter/approximation hides  $B$ ; supplied-budget results require it. Native costs are uncalibrated; caches, instrumentation and compilation limit timings. Archived timeouts, counterexamples and reversed timings remain. Under  $W + L + 8\kappa Q$ , three modes improve 2/96 cells; 94 tie. A separate 72-cell JMH study (four payloads, two bin layouts, nine wrappers) fits nonnegative work/validation/callback/premium prices in only 2/8 settings at 20% maximum error, including cached validation. All fitted premiums are zero: 0/72 admitted sensitivity roots improve; 1,080 lie outside the model. All-budget compilation can exceed requested-value cost. Valkey fixed-digest materialization beats reader repair in time/transfer in all 16 cells; maximum command times are mixed. Larger interfaces therefore matter. Finite/AI checks are neither mechanical proofs nor independent reproduction; human benefit is unmeasured.

## 12 Conclusion

An exact filter exposes all safe completing modes without future body data. Every disciplined caller on every DAG preserves optimal protection within the sharp work factor  $(3r + 2)/(2r + 2)$  of the full simultaneous-contract optimum. Independent jobs admit exact work; mismatch/validation charges reveal the future-fee boundary. Conditional Java transfer and adverse native comparisons delimit application.

## Data Availability

The anonymous artifact [48] provides code, inputs, proofs, outcomes/corrections, licenses, 60 fixed-evidence replay stages and separate native rebuild instructions. Stage60 re-executes the caller-approximation checks; stages58/59 cover the fee extensions.

## References

- [1] Michael R. Garey and David S. Johnson. 1976. Scheduling Tasks with Nonuniform Deadlines on Two Processors. *Journal of the ACM* 23, 3, 461–467. Theorem 2 (pp. 465–466) concerns one processor. <https://doi.org/10.1145/321958.321967>.
- [2] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. 2018. Build Systems à la Carte. *PACMPL* 2, ICFP, Article 79, 1–29. <https://doi.org/10.1145/3236774>. Author version: <https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf>.
- [3] Ruy Ley-Wild, Umut A. Acar, and Matthew Fluet. 2009. A Cost Semantics for Self-Adjusting Computation. In *POPL*, 186–199. <https://doi.org/10.1145/1480881.1480907>. Author version: <https://ttic.uchicago.edu/~pl/sa-sml/popl09.pdf>.
- [4] Rachid Guerraoui, Maurice Herlihy, and Bastian Pochon. 2005. Toward a Theory of Transactional Contention Managers. In *PODC*. Author version. <https://lpdwww.epfl.ch/upload/documents/publications/neg--509904679f181-pochon.pdf>.
- [5] Nuno Diegues and Paolo Romano. 2014. Self-Tuning Intel Transactional Synchronization Extensions. In *ICAC*, 209–219. <https://www.usenix.org/conference/icac14/technical-sessions/presentation/diegues>. Errata.
- [6] H. T. Kung and John T. Robinson. 1981. On optimistic methods for concurrency control. *ACM Transactions on Database Systems* 6, 2, 213–226. <https://doi.org/10.1145/319566.319567>.
- [7] Linux contributors. 2024. Linux v6.12, fs/d\_path.c and fs/hostfs/hostfs\_kern.c, commit adc218676eef25575469234709c2d87185ca223a. Accessed September 11, 2026. [https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/d\\_path.c](https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/d_path.c).
- [8] Linux contributors. 2024. Linux v6.12, fs/dcache.c: name snapshots, d\_exchange, and tmpfile initialization. Accessed September 11, 2026. <https://github.com/torvalds/linux/blob/adc218676eef25575469234709c2d87185ca223a/fs/dcache.c>.
- [9] Waiman Long. 2013. dcache: Translating dentry into pathname without taking rename\_lock. Linux commit 232d2d60aa5469bb097f55728f65146bd49c1d25. <https://github.com/torvalds/linux/commit/232d2d60aa5469bb097f55728f65146bd49c1d25>.
- [10] .NET Foundation. 2025. *Roslyn source*, commit 6c4a46a31302167b425d5e0a31ea83c9a9aa1d09. Workspace.cs, Solution.cs, SolutionCompilationState.cs and SolutionState.cs. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/tree/6c4a46a31302167b425d5e0a31ea83c9a9aa1d09>.
- [11] .NET Foundation contributors. 2020. Add SetCurrentSolution with solution transformation, pull request 42228. Source review on replayability and event ordering. Accessed September 10, 2026. <https://github.com/dotnet/roslyn/pull/42228>.
- [12] Shie Mannor, Ofir Mebel, and Huan Xu. 2012. Lightning Does Not Strike Twice: Robust MDPs with Coupled Uncertainty. In *ICML*. <https://icml.cc/2012/papers/215.pdf>.
- [13] Avi Federgruen and Henri Groenevelt. 1986. The Greedy Procedure for Resource Allocation Problems: Necessary and Sufficient Conditions for Optimality. *Operations Research*. [https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen\\_greedy\\_procedure.pdf](https://business.columbia.edu/sites/default/files-efs/pubfiles/4071/federgruen_greedy_procedure.pdf).
- [14] Dimitris Bertsimas and Melvyn Sim. 2004. The Price of Robustness. *Operations Research* 52, 1, 35–53. <https://www.mit.edu/~dbertsim/papers/Robust%20Optimization/The%20price%20of%20Robustness.pdf>.
- [15] Pavol Černý, Krishnendu Chatterjee, Thomas A. Henzinger, Arjun Radhakrishna, and Rohit Singh. 2011. Quantitative Synthesis for Concurrent Programs. In *CAV*, 243–259. [https://doi.org/10.1007/978-3-642-22110-1\\_20](https://doi.org/10.1007/978-3-642-22110-1_20). Author publication page and complexity erratum: <https://www.microsoft.com/en-us/research/publication/quantitative-synthesis-for-concurrent-programs/>.
- [16] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2015. Optimizing Solution Quality in Synchronization Synthesis. *arXiv preprint arXiv:1511.07163v1*. <https://arxiv.org/abs/1511.07163>.
- [17] Pavol Černý, Edmund M. Clarke, Thomas A. Henzinger, Arjun Radhakrishna, Leonid Ryzhyk, Roopsha Samanta, and Thorsten Tarrach. 2017. From non-preemptive to preemptive scheduling using synchronization synthesis. *Formal Methods in System Design* 50, 97–139. <https://doi.org/10.1007/s10703-016-0256-5>.
- [18] Oracle. 2021. ConcurrentHashMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentHashMap.html>.
- [19] Oracle. 2021. ConcurrentMap, Java SE 17 API Specification. Accessed September 7, 2026. <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/concurrent/ConcurrentMap.html>.
- [20] OpenJDK. 2026. ConcurrentHashMap.java, jdk17u source, commit 982d53c227943a511e8d61694feced764b71a6c1. Accessed September 7, 2026. <https://github.com/openjdk/jdk17u/blob/982d53c227943a511e8d61694feced764b71a6c1/src/java.base/share/classes/java/util/concurrent/ConcurrentHashMap.java>.



- [21] Krzysztof Postek, Ward Romeijnders, and Wolfram Wiesemann. 2025. Technical Note: Multi-Stage Robust Mixed-Integer Programming. *Operations Research* 73, 6, 3345–3358. <https://doi.org/10.1287/opre.2023.0520>. Online appendix; author code.
- [22] Anonymous Authors. 2026. *Compiling Optimal Retry Policies for Dependent Transformations: Research Artifact*. Version 5 with publication supplement, commit e0e0110. <https://github.com/research-artifact-archive/research-artifact/tree/e0e011023c0591bbbf422390ad8d29785ad0023f>.
- [23] Anonymous Authors. 2026. *Retry Synthesis with Bounded Calls and Protected Work: Research Artifact*. Total-work, comparison-free and unit-guard precedence, and call-accounting supplement, commit b719eb7. <https://github.com/research-artifact-archive/research-artifact/tree/b719eb7a6514d7f3a47f2a3af21e43b4ffd4699f>.
- [24] Hagit Attiya, Leah Epstein, Hadas Shachnai, and Tami Tamir. 2006. Transactional Contention Management as a Non-Clairvoyant Scheduling Problem. In *PODC*. <https://cs.idc.ac.il/~tami/Papers/podc06.pdf>.
- [25] Dan Alistarh, Syed Kamran Haider, Raphael Kübler, and Giorgi Nadiradze. 2018. The Transactional Conflict Problem. *arXiv preprint arXiv:1804.00947v1*. <https://arxiv.org/abs/1804.00947>.
- [26] Free Software Foundation. 2025. GNU C Library 2.42: x86 lock elision, revision d2097651. `elision-lock.c` and `elision-conf.c`. <https://github.com/bminor/glibc/tree/d2097651cc57834dbfcaa102ddfacae0d86cfb66/sysdeps/unix/sysv/linux/x86>. Official tunables: [https://www.gnu.org/software/libc/manual/html\\_node/Elision-Tunables.html](https://www.gnu.org/software/libc/manual/html_node/Elision-Tunables.html).
- [27] OpenJDK. 2025. HotSpot 21.0.8+9 RTM monitor code, revision 54f20959. `c2_MacroAssembler_x86.cpp` and `globals_x86.hpp`. <https://github.com/openjdk/jdk21u/tree/54f2095960f01d957f2335cfa0defb956e13a2c/src/hotspot/cpu/x86>.
- [28] GNU Compiler Collection. n.d. x86 Transactional Memory Intrinsics. Accessed September 10, 2026. <https://gcc.gnu.org/onlinedocs/gcc/x86-transactional-memory-intrinsics.html>.
- [29] Aditya Dave, Nishanth Venkatesh, and Andreas A. Malikopoulos. 2024. Approximate Information States for Worst-Case Control and Learning in Uncertain Systems. *arXiv preprint arXiv:2301.05089v2*. <https://arxiv.org/abs/2301.05089>.
- [30] Matthew A. Hammer, Khoo Yit Phang, Michael Hicks, and Jeffrey S. Foster. 2014. Adapton: Composable, Demand-Driven Incremental Computation. In *PLDI*. <https://doi.org/10.1145/2594291.2594324>. Author version: <https://www.cs.tufts.edu/~jfofster/papers/pldi14.pdf>.
- [31] Mike Ulrich. 2016. Addressing Cascading Failures. In *Site Reliability Engineering*, Chapter 22, Retries. <https://sre.google/sre-book/addressing-cascading-failures/>.
- [32] Envoy contributors. n.d. CircuitBreakers.Thresholds.RetryBudget. Accessed September 10, 2026. [https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit\\_breaker.proto.html](https://www.envoyproxy.io/docs/envoy/latest/api-v3/config/cluster/v3/circuit_breaker.proto.html).
- [33] Adam Welc, Bratin Saha, and Ali-Reza Adl-Tabatabai. 2008. Irrevocable Transactions and their Applications. In *SPAA*, 285–296. <https://doi.org/10.1145/1378533.1378584>.
- [34] Eugene L. Lawler. 1973. Optimal Sequencing of a Single Machine Subject to Precedence Constraints. *Management Science* 19, 5, 544–546. <https://doi.org/10.1287/mnsc.19.5.544>. Author technical report ERL-M327 (1971): <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1971/Archive/ERL-m-327.pdf>.
- [35] Christoph Dürr, Thomas Erlebach, Nicole Megow, and Julie Meißner. 2020. An Adversarial Model for Scheduling with Testing. Author preprint arXiv:1709.02592v3. <https://arxiv.org/abs/1709.02592v3>.
- [36] Christoph Damerius, Peter Kling, Minming Li, Chenyang Xu, and Ruilong Zhang. 2023. Scheduling with a Limited Testing Budget: Tight Results for the Offline and Oblivious Settings. Author preprint arXiv:2306.15597v1. <https://arxiv.org/abs/2306.15597v1>.
- [37] Fanny Dufossé, Christoph Dürr, Noël Nadal, Denis Trystram, and Óscar C. Vázquez. 2021. Scheduling with a Processing Time Oracle. Author preprint arXiv:2005.03394v3. <https://arxiv.org/abs/2005.03394v3>.
- [38] Felix Buld and Andreas S. Schulz. 2025. Scheduling with Testing: Competitive Algorithms for Minimizing the Total Weighted Completion Time in the Adversarial Model. In *Frontiers of Algorithmics, IJTCS-FAW 2025*, LNCS 15828, 64–77. [https://doi.org/10.1007/978-981-96-8312-3\\_5](https://doi.org/10.1007/978-981-96-8312-3_5). Full author version: <https://arxiv.org/abs/2606.25166>.
- [39] Susanne Albers and Alexander Eckl. 2021. Explorable Uncertainty in Scheduling with Non-Uniform Testing Times. Author preprint arXiv:2009.13316v2; first version 2020. <https://arxiv.org/abs/2009.13316v2>.
- [40] Susanne Albers and Maximilian Janke. 2021. Online Makespan Minimization with Budgeted Uncertainty. In *WADS*. [https://doi.org/10.1007/978-3-030-83508-8\\_4](https://doi.org/10.1007/978-3-030-83508-8_4). Author version: <https://maximilian-janke.de/data/BudgetedUncertainty.pdf>.
- [41] Beng-Hong Lim and Anant Agarwal. 1994. Reactive Synchronization Algorithms for Multiprocessors. In *ASPLOS VI*, 25–35. Author version: <https://groups.csail.mit.edu/cag/pub/papers/pdf/reactive.pdf>.
- [42] Takayuki Usui, Reimer Behrendts, Jacob Evans, and Yannis Smaragdakis. 2010. Adaptive Locks: Combining Transactions and Locks for Efficient Concurrency. *Journal of Parallel and Distributed Computing*. <https://doi.org/10.1016/j.jpdc.2010.02.006>. Author version: <https://people.cs.umass.edu/~yannis/adaptive-jpdc.pdf>.
- [43] Michael Luby, Alistair Sinclair, and David Zuckerman. 1993. Optimal Speedup of Las Vegas Algorithms. *Information Processing Letters* 47, 4, 173–180. Author version: <https://www.cs.utexas.edu/~diz/pubs/speedup.pdf>.



- [44] Walther Maldonado, Patrick Marlier, Pascal Felber, Julia Lawall, Gilles Muller, and Etienne Rivière. 2011. Deadline-Aware Scheduling for Software Transactional Memory. In *DSN*. Author version, Section III.D. <https://who.paris.inria.fr/Gilles.Muller/papers/dsn2011.pdf>.
- [45] Roderick Bloem, Bettina Könighofer, Robert Könighofer, and Chao Wang. 2015. Shield Synthesis: Runtime Enforcement for Reactive Systems. Extended author version arXiv:1501.02573v2. <https://arxiv.org/abs/1501.02573v2>.
- [46] Yao Feng, Jun Zhu, André Platzer, and Jonathan Laurent. 2025. Adaptive Shielding via Parametric Safety Proofs. *PACMPL* 9, OOPSLA1. <https://doi.org/10.1145/3720450>. Author version: <https://arxiv.org/abs/2502.18879>.
- [47] Filip Cano, Thomas A. Henzinger, and Konstantin Kueffner. 2026. Shielding for Higher-Order Safety. Author version arXiv:2608.03662v1. <https://arxiv.org/abs/2608.03662v1>.
- [48] Anonymous Authors. 2026. *Resource Contracts for Completing Optimistic Computations: Research Artifact*. Sharp DAG caller approximation, charged residual and Java implementation supplement, commit 759160f. <https://github.com/research-artifact-archive/research-artifact/tree/759160fa128d0f2653d3195ea0d9b2289f4198f8>.
- [49] Ronald L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal on Applied Mathematics* 17, 2, 416–429. Theorem 1, pp.419–420. [https://fanchung.ucsd.edu/ron/papers/69\\_02\\_multiprocessing.pdf](https://fanchung.ucsd.edu/ron/papers/69_02_multiprocessing.pdf).
- [50] Adriaan ter Mors, Chetan Yadati, Cees Witteveen, and Yingqian Zhang. 2010. Coordination by Design and the Price of Autonomy. *Autonomous Agents and Multi-Agent Systems* 20, 308–341. <https://doi.org/10.1007/s10458-009-9086-9>.